

函数级代码生成的预算分档编排：统一协议、机制证据与成本前沿

Budget-Tiered Test-Time Orchestration for Function-Level Code Generation

薛沁晨

摘要

函数级代码生成系统在部署时同时受到接口计费、上下文长度与调用预算约束，核心问题是如何把有限测试时计算投向最能提升通过率的环节。本文提出预算分档编排协议，在统一执行器、统一判定口径和统一令牌记账下，将流程组织为单步、单步 + 修补、分流重链与救援、级联四个可比较档位，用以刻画不同预算投向形成的成本-收益前沿。基于 HumanEval 164 题与 MBPP 清洗集 427 题的实验表明，以 `claude-sonnet-4-6` 为主模型时，MBPP 通过率由 82.51% 提升至 91.72%，HumanEval 由 97.56% 提升至 99.80%。机制实验进一步显示，结构难度分能够作为预算分配先验，固定轻/重路径多重集对照揭示结构对齐在 MBPP 上带来额外通过样本，压缩失败回注在五随机种子下保持相近通过率并降低平均令牌。累计收益率 ρ 表明，单步 + 修补具有最高边际效率，级联档提供最高通过率上限。本文为函数级代码生成的测试时编排提供了一套可复现、可解释、可记账的实验口径。

关键词：函数级代码生成；测试时计算；预算分档；失败修补；级联编排；成本收益前沿

Abstract

Function-level code generation systems are constrained in deployment by API cost, context length, and call budgets. The key question is how to allocate limited test-time computation to stages that most improve correctness. This paper proposes a budget-tiered orchestration protocol that fixes one executor, one correctness criterion, and one token-accounting pipeline, then organizes generation into four comparable tiers: single-step generation, repair after failure, difficulty-aware rescue, and cascaded orchestration. Experiments on HumanEval-164 and MBPP-sanitized-427 with `claude-sonnet-4-6` show that the pass rate rises from 82.51% to 91.72% on MBPP and from 97.56% to 99.80% on HumanEval. Mechanistic results show that structural difficulty scores serve as useful budget priors, fixed light/heavy-path multiset controls expose the effect of structural alignment, and compressed failure feedback preserves comparable accuracy with lower average token use across five seeds. The cumulative gain-rate frontier ρ indicates that single-step repair delivers the strongest marginal efficiency, while cascaded orchestration reaches the highest pass-rate ceiling. The protocol offers a reproducible, interpretable, and cost-accountable basis for studying test-time orchestration in code generation.

Keywords: function-level code generation; test-time computation; budget tiering; failure repair; cascade orchestration; cost-benefit frontier

1 引言

大语言模型已经成为函数级代码生成系统的核心组件。HumanEval 与 MBPP 等基准表明，强模型能够在相当比例的短函数任务上通过单次生成得到可运行实现 [1, 2]。真实部署进一步关心接口计费、上下文长度与调用预算共同约束下的系统编排。对同一失败样本，重新采样、执行反馈修补、结构化重链、候选扩展与执行筛选都会改变通过率和成本之间的关系 [3, 4, 5, 6, 7]。

现有研究已经分别验证了若干有效组件：执行反馈能够修补错误程序，候选扩展与筛选能够提高通过率，反思或批判式提示能够改善复杂任务。然而这些组件在工程系统中往往以多个开关叠加出现，若执行器、统计口径或默认提示模板不同，最终收益很难归因到预算组织本身。本文关注的正是这一问题：在同一实现中逐级解锁不同测试时计算投向，同时记录通过率、调用次数与令牌成本。

本文将这一口径称为**预算分档编排**。协议把函数级代码生成流程组织为四个语义明确的档位：单步档作为最低成本基线；单步 + 修补档在失败后引入执行反馈；分流 + 重链与救援档依据结构难度先验选择重路径，并将失败信息压缩回注；级联档在高预算下扩展候选集并用执行结果筛选。四档共享同一外层闭环，差异只在新增预算投向，使成本-收益关系能够被直接比较。

该协议同时给出机制层面的解释路径。结构难度分承担预算排序作用，用于把更复杂的样本优先送入重路径；固定轻/重路径多重集的随机对照用于隔离覆盖率和结构对齐；压缩回注与完整回溯的对照刻画反馈形式对修补收益和令牌成本的影响；级联消融进一步区分候选扩展和后处理精修的贡献。这样，端到端结果和模块机制可以在同一实验口径下相互印证。

本文的主要贡献如下：

1) **协议贡献**：提出面向函数级代码生成的预算分档编排协议，在统一执行器、统一判定与统一记账下比较单步、修补、分流救援与级联四类预算形态，将测试时计算从调用次数多少转化为预算投向差异。

2) **机制贡献**：围绕分流、反馈与级联三个环节给出机制实验。结构难度分被建模为预算分配先验，固定轻/重路径多重集对照检验结构对齐效果，压缩

反馈与完整回溯的五随机种子对照评估回注形式。

3) **评测贡献**：在 HumanEval 与 MBPP 双基准上报告四档三随机种子主结果，并同时给出通过率、平均调用次数、总令牌与经验收益率 ρ 。实验显示，MBPP 上分档收益更充分，HumanEval 上主要体现为高位残差收束，二者共同展示了预算编排在不同任务分布下的价值。

全文组织如下。第 2 节梳理代码生成评测、执行反馈修补与测试时扩展相关工作；第 3 节给出问题定义、预算档位和成本记账口径；第 4 节描述分流、救援与级联的实现机制；第 5 节报告主结果与消融；第 6 节讨论部署含义、机制边界与局限；第 7 节总结全文。

2 相关工作

2.1 函数级代码生成评测

函数级代码生成通常以题面、函数签名和单元测试评测模型能否生成可执行程序。HumanEval[1] 与 MBPP[2] 是该方向最常用的基准，前者规模较小、题面相对集中，后者覆盖更广的 Python 任务。随后，APPS、CodeXGLUE、DS-1000、EvalPlus 与 LiveCodeBench 等工作进一步扩展了任务难度、领域范围和测试覆盖 [8, 19, 11, 9, 10]。

上述基准的共同价值在于提供可执行判定，使代码生成研究能够超越文本相似度指标。本文沿用 HumanEval 与 MBPP，是因为二者均为函数级任务，适合在同一执行器下比较测试时编排策略。除通过率外，本文同步记录调用次数与令牌成本，以反映接口部署中的实际约束。

2.2 执行反馈驱动的程序修补

执行反馈为代码生成提供了自然的外部验证信号。Self-Debug、Reflexion、CRITIC、Self-Refine、Agent-Coder 与 CodeT 等工作表明，错误信息、测试结果或工具反馈可以被回注到模型上下文中，用于修补失败程序或生成新的候选实现 [4, 5, 6, 13, 12, 20]。这一路线的核心问题在于反馈如何进入下一轮生成。完整回溯包含较多运行环境文本，会增加令牌成本；压缩反馈则把错误类型、关键片段和修补方向组织成更短的约束。

本文继承执行反馈修补的基本思想，并将其放

入预算分档协议中考察：反馈修补是中预算档位的主要计算投向。本文单独比较压缩反馈与完整回溯，在相同流程下观察通过率与令牌成本的差异。

2.3 测试时计算扩展与候选筛选

另一条相关路线是通过测试时计算扩展提高覆盖率。AlphaCode 展示了大规模候选生成与筛选在竞赛级代码生成中的作用 [3]，后续测试时扩展工作进一步讨论候选数量、筛选策略与成本之间的关系 [7]。这些方法说明，在模型能力既定时，额外候选可以提高命中可行解的概率，同时带来调用次数和令牌成本的增长。

本文的级联档与该思路一致，即把高预算主要投入候选扩展和执行筛选。通过关闭候选扩展、限制精修次数等消融，本文区分主增益来源与边际后处理，并用经验收益率 ρ 描述追加预算的成本前沿。与强调单一技巧提升的工作不同，本文进一步关注同一实现下的预算组织问题：当分流、修补、候选扩展和精修同时可用时，系统需要以可比较、可复现和可记账的方式组织它们。因此，本文先在固定评测口径下给出端到端结果，再用机制实验解释预算投向的收益来源。

3 问题定义与协议

本节给出本文研究对象的形式化定义。本文把测试时计算视为可记账的资源分配问题：在同一代码生成任务集合上，给定一个预算档位，系统决定是否继续调用模型、如何使用执行反馈、是否扩展候选集，并在统一执行器下记录收益与成本。

3.1 函数级代码生成与统一判定

设函数级代码生成数据集为

$$\mathcal{D} = \{x_i\}_{i=1}^N, \quad (1)$$

其中 x_i 包含题面、函数签名以及测试接口。给定预算档位 b ，编排器生成一个或多个候选程序，最终提交候选 $y_i^{(b)}$ 至执行器 E ，得到二值判定

$$z_i^{(b)} = E(y_i^{(b)}) \in \{0, 1\}. \quad (2)$$

通过率定义为 $p_b = N^{-1} \sum_i z_i^{(b)}$ 。除通过率外，本文对每题记录模型调用次数 $c_i^{(b)}$ 与令牌消耗 $t_i^{(b)}$ ，并汇总得到平均调用次数与总令牌。这里的关键约束

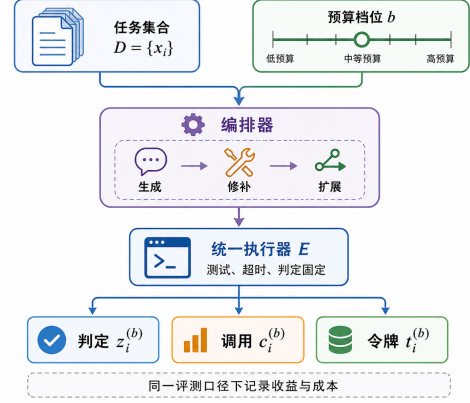


图 1: 统一预算记账协议

是：所有档位共享同一执行器、同一超时与同一测试判定口径，差异仅来自预算档位允许的后续动作。

这一设定把问题从某一提示是否更好，转化为某一预算形态能在相同统计口径下带来多少收益。后文主结论以同协议内的四档结果为核心证据，跨模型结果与细粒度消融用于解释机制来源。

图 1 概括了本文的记账口径：预算档位只改变编排器允许采取的动作，判定与成本记录保持统一。

3.2 预算档位与语义约束

预算分档协议将测试时编排限制为四个语义清晰的档位。档位越高，解锁的计算投向越丰富：从单次生成，到失败修补，再到难度分流下的重路径，最后到候选扩展与执行筛选。表 1 给出语义档位与仓库实现模式之间的对应关系。其中第三档在双基准上共用 `difficulty_aware_plus` 实现，命令行预算别名仅作为复现入口，正文仍按语义档位定义；第一档日志别名 `base_direct` 与 `single_step` 同义。

表 1: 预算语义档位与实现模式映射

语义档位	实现模式	允许的主要计算投向
单步	<code>single_step</code>	单次生成与执行判定；作为最低成本基线
单步 + 修补	<code>single_step_repair</code>	在单次失败后使用执行反馈做有限修补
分流 + 重链与救援	<code>difficulty_aware_plus</code>	依据结构难度进入轻/重路径，重路径启用结构化主链与失败后救援
级联	<code>workflow_cascade</code>	多候选扩展、执行筛选以及有限精修/融合

该映射有两个作用。首先，它统一模式名与预算档位，减少解释歧义；其次，它为成本解释提供

统一坐标，使通过率上升和令牌增长能够放在同一成本-收益框架下讨论。

3.3 成本记账与经验收益率

真实部署中，调用次数只反映部分成本。一次长上下文修补可能比多次短调用更昂贵，级联阶段的候选组织也会推高令牌消耗。因此，本文以总令牌作为主成本代理，以平均调用次数作为辅助指标。若某档 b 相对前一档 $b-1$ 的通过率增量为 $\Delta p = p_b - p_{b-1}$ ，额外令牌为 $\Delta t = t_b - t_{b-1}$ ，则定义经验性边际收益率

$$\rho = \frac{\Delta p}{\Delta t}. \quad (3)$$

为便于跨档比较，实验节还报告相对单步档的累计收益率

$$\rho_b^{(\text{cum})} = \frac{p_b - p_{\text{single}}}{t_b - t_{\text{single}}}. \quad (4)$$

这两个量用于解释新增令牌带来的通过率收益，帮助判断不同档位的边际效率。

3.4 证据层级

本文按三层组织证据。第一层是双基准四档三随机种子主结果，用于刻画预算分档的整体前沿。第二层是多模型单随机种子扫描，用于观察不同基础模型下的档位形态。第三层是机制消融，包括阈值扫描、流程开关、反馈形态和随机路由对照，用于回答预算主要花在哪里、收益来自哪个模块。

这一组织方式贯穿全文，使主结果、跨模型形态和机制解释各自承担清晰角色。

4 预算分档编排方法

本节描述四档协议在实现层面的组织方式。系统围绕同一个外层闭环运行：读入任务、生成候选、执行测试、记录结果；若失败，则在当前预算档位允许的范围内追加修补、重链、救援或级联。整体流程见图 2。

4.1 总体流程

对每个任务 x_i ，系统首先生成单步候选并交由执行器判定。候选通过后流程结束；候选失败后，低档位直接结束或进入轻量修补，高档位根据结构难

度、执行反馈与候选池状态决定后续动作。这样的组织把新增预算集中在**单步未能覆盖的残差样本**上。

四档的差异可以概括如下。单步档只做一次生成，是最低成本基线；单步 + 修补档在失败后追加有限修补调用，检验执行反馈的边际价值；分流 + 重链与救援档先判断样本是否进入重路径，再使用结构化提示、错误压缩与候选救援处理复杂失败；级联档进一步扩展候选集合，并用执行筛选和可选精修组织高预算搜索。

算法 1 给出单个样本上的预算分档编排逻辑。算法只刻画档位间的控制流，结构难度打分、失败反馈压缩与级联候选扩展的具体实现分别在后续小节展开。

4.2 结构难度路由

难度路由用于在进入重路径之前完成一次粗粒度预算筛分。它承担**预算分配先验**的角色：结构上更复杂的题目更需要额外上下文、结构化提示或候选扩展，因而优先进入重路径。

图 3 展示了结构分数到轻/重路径的门控关系。

记样本 x_i 的结构难度分数为 s_i 。实现中的 v2 打分器将分数拆为题面复杂度、签名/接口复杂度、测试复杂度、边界条件密度、题面歧义以及有界体量项：

$$s_i = p_i + \sigma_i + \tau_i + b_i + a_i + e_i. \quad (5)$$

其中 $p_i, \sigma_i, \tau_i, b_i, a_i$ 分别对应题面、签名、测试、边界与歧义五类子项， e_i 为由题面长度、测试长度、断言数量等构成的有界体量项。各子项由固定启发式规则加权并裁剪上限，总分由题面结构确定。

路由决策写为

$$r_i = \mathbb{I}(s_i \geq \tau_{\text{eff}}), \quad (6)$$

其中 τ_{eff} 为配置中的结构阈值，主实验第三档默认使用 1.6。 $r_i = 1$ 表示进入重路径主链， $r_i = 0$ 表示停留在轻路径。阈值越低，重路径覆盖率越高，通常带来更高成本和更高通过率上限；阈值越高，成本下降，同时覆盖样本减少。该关系由第 5 节的阈值消融检验。

为分离重路径覆盖率与结构对齐的影响，仓库实现了 `random_permute_matched` 对照：先按结构分得到本批样本的轻/重路径标签多重集，再固定随机种子置换标签，使重路径数量与默认结构路由完全一致，但题目与重路径槽位的对应被打乱。该对

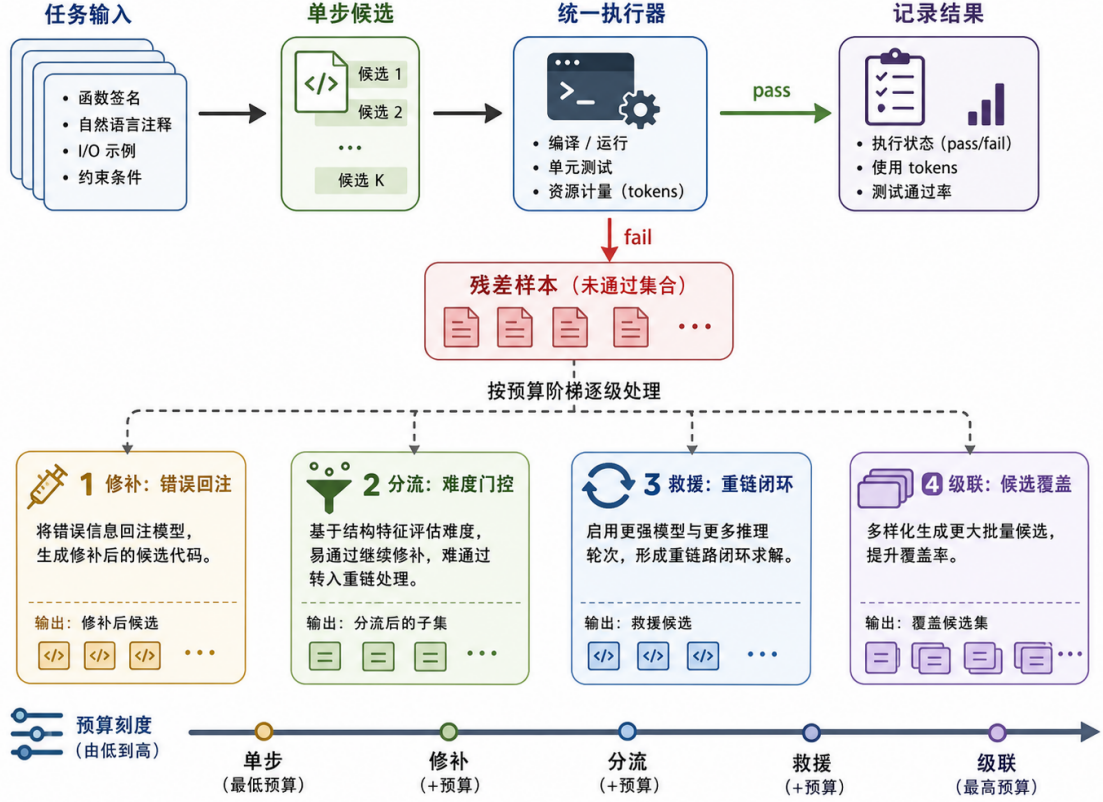


图 2: 预算分档编排整体框架

照刻画固定覆盖率下的结构对齐效果。

4.3 失败反馈与救援链

执行失败后，系统把失败信息转化为下一轮生成可利用的局部约束。其基本思路与 Self-Debug、Reflection 和 CRITIC 等执行反馈工作一致 [4, 5, 6]，本文进一步考察反馈形式对令牌成本与修补收益的影响。

在分流 + 重链与救援档中，系统先根据执行器返回的 `check_correctness` 字符串判断失败形态。该判断由规则函数完成：未闭合字符串、语法非法等更倾向于替换生成；缺参、参数数量不匹配、名称未定义或缩进错误等归入修补；其余进入更保守的替换或救援路径。当单步与结构化双路径均失败时，系统比较二者错误标签，以决定后续修补轮数。

压缩后的回注包记为

$$\phi(e_i^{(1)}) = \text{Concat}(\text{错误类型, 截断摘要, 规则片段, 定向修补提示}). \quad (7)$$

其中 $e_i^{(1)}$ 为第一次失败反馈。第二轮上下文可写作

$$\tilde{x}_i^{(2)} = \text{Concat}(x_i, \phi(e_i^{(1)})), \quad (8)$$

并据此生成新候选

$$y_i^{(2)} \sim p_{\theta}(\cdot | \tilde{x}_i^{(2)}). \quad (9)$$

该表达式说明失败信息如何被裁剪并进入下一轮上下文。图 4 展示了这一闭环。

本文在实验中比较压缩回注与完整回溯两种形态。五随机种子结果显示，压缩回注保持相近通过率，同时平均令牌更低，因此作为默认回注形式。

4.4 高预算级联

级联档把新增预算转向候选空间覆盖。其流程包括三层：候选扩展层生成额外候选，执行筛选层用统一测试选择可行实现，后处理层对进入候选池的样本做有限精修或融合。该设计与测试时扩展的基本思想一致，即通过候选扩展提高命中可行解的概率 [3, 7]。

实现上，级联主链按基线候选、额外候选扩展、执行筛选和可选后处理的顺序运行。额外候选阶段的生成次数受全局上限、路由表与执行 top-k 共同约束；实际执行数为生成数与 top-k 的较小值。可行解指在统一执行器下通过官方单元测试的候选实现。若候选扩展已产生可行解，精修只作为精度-成

算法 1: 预算分档编排协议

输入: 任务 x_i , 预算档位 b , 模型 p_θ , 执行器 E , 结构阈值 τ_{eff}

输出: 最终候选 y_i , 判定 z_i , 调用次数 c_i , 令牌消耗 t_i

where: $\text{SINGLE} \prec \text{REPAIR} \prec \text{ROUTE} \prec \text{CASCADE}$ defines the budget order

```
1  $y^* \leftarrow \text{Generate}(p_\theta, x_i);$ 
2  $(z^*, e^*) \leftarrow E(y^*);$  update  $c_i, t_i;$  // single-step baseline
3 if  $z^* = 1$  or  $b = \text{SINGLE}$  then
4   return  $(y^*, z^*, c_i, t_i);$ 
5 end
6 if  $b \succeq \text{REPAIR}$  then
7    $\tilde{x}_i \leftarrow \text{Concat}(x_i, \phi(e^*));$  // compressed feedback
8    $y \leftarrow \text{Generate}(p_\theta, \tilde{x}_i);$   $(z, e) \leftarrow E(y);$  update  $c_i, t_i;$ 
9   if  $z = 1$  then  $(y^*, z^*, e^*) \leftarrow (y, z, e);$ 
10  if  $z^* = 1$  or  $b = \text{REPAIR}$  then
11    return  $(y^*, z^*, c_i, t_i);$ 
12  end
13 end
14  $s_i \leftarrow \text{Difficulty}(x_i);$   $r_i \leftarrow \mathbb{I}(s_i \geq \tau_{\text{eff}});$ 
15 if  $r_i = 1$  then
16    $(y, z, e) \leftarrow \text{HeavyChain}(x_i, e^*);$  update  $c_i, t_i;$ 
17   if  $z = 1$  then  $(y^*, z^*, e^*) \leftarrow (y, z, e);$ 
18 end
19 if  $z^* = 0$  then
20    $(y, z, e) \leftarrow \text{Rescue}(x_i, e^*);$  update  $c_i, t_i;$ 
21   if  $z = 1$  then  $(y^*, z^*, e^*) \leftarrow (y, z, e);$ 
22 end
23 if  $b = \text{CASCADE}$  and  $z^* = 0$  then
24    $\mathcal{Y} \leftarrow \text{ExpandCandidates}(x_i, y^*, e^*);$ 
25    $(y^*, z^*) \leftarrow \text{SelectByExecutor}(\mathcal{Y}, E);$  update  $c_i, t_i;$ 
26 end
27 return  $(y^*, z^*, c_i, t_i);$ 
```

本微调。

第 5 节的级联消融显示, 额外候选扩展是高预算档的主增益来源; 精修相关开关更多体现为令牌与少量精度之间的折中。级联档的核心是用候选覆盖换取更高上限。

4.5 实现不变式与可比性控制

为保证不同预算档位之间的比较只反映编排动作差异, 本文在实现中固定三类不变式。第一, 所有档位共享同一执行器、测试接口、超时设置和通过判定函数, 避免把评测口径差异混入预算收益。第二, 结构难度分数只由题面、签名和测试结构等预

执行信息计算, 不使用执行结果或人工标签; 随机置换路由对照进一步把重路径覆盖率与结构对齐效果分开。第三, 失败回注只使用执行器返回的错误信息及规则化摘要, 不引入参考答案或额外人工修正。上述约束使四档协议能够作为同一测试时计算空间中的可记账比较, 而不是彼此独立的提示模板集合。

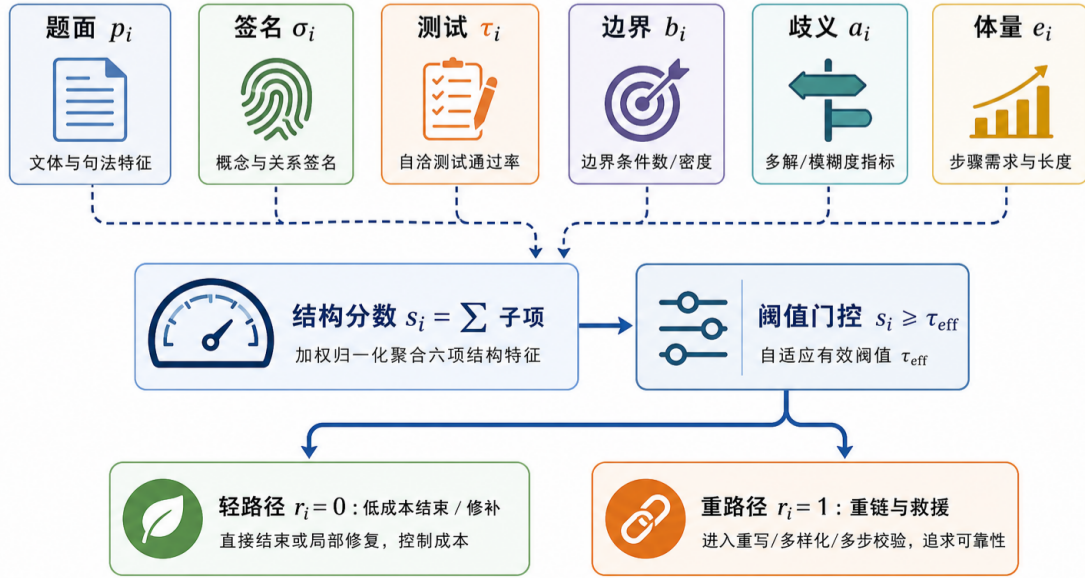


图 3: 结构难度路由决策

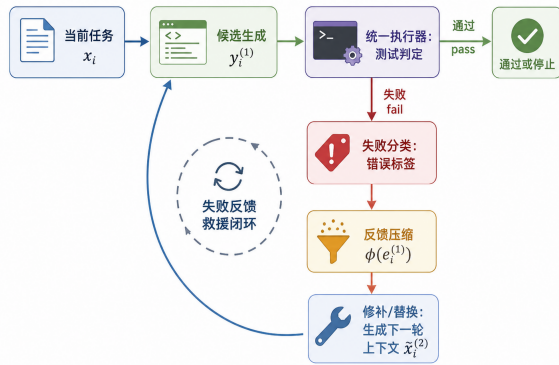


图 4: 失败反馈救援闭环

通过率衡量任务正确性，调用次数反映调度复杂度，总令牌作为接口成本的主代理。表 2 汇总本文的公平性约束。

表 2: 实验协议约束

约束项	本文约束
数据口径	MBPP 清洗集 427 + HumanEval 164
模型与版本	主结果统一 <code>claude-sonnet-4-6</code> ；多模型结果单独列出
随机性控制	主表四档随机种子统计；E2 反馈形态对照五随机种子；多重集随机路由对照为随机种子 42 单次配对
执行与判定	统一执行器、超时、测试调用方式与通过判定口径
比较轴线	主结论以同协议内预算分档双基准主结果为准；多模型表和消融表解释形态与机制
成本指标	同时报告通过率、平均调用次数、总令牌

5 实验

5.1 评测协议与数据集

本文在 HumanEval 164 题与 MBPP 清洗集 427 题上评测预算分档编排。两者均属于函数级代码生成任务，能够使用同一执行器和同一通过判定口径；差异在于任务分布和动态范围。HumanEval 在当前强模型设定下更接近饱和和区间，适合观察残差收束；MBPP 题面更杂、规模更大，更适合观察预算扩展带来的成本-收益曲线。

主结果统一使用 `claude-sonnet-4-6`，并对四个预算档位报告三随机种子（42、78、132）的均值与样本标准差。多模型结果和多数细粒度消融采用随机种子 42，用于补充观察档位形态和模块机制。

评价指标包括通过率、平均调用次数与总令牌。

表 2 的约束确保不同档位只在预算动作上发生差异：执行器、判定函数和成本口径保持一致，后续通过率和令牌变化才能解释为编排策略带来的收益与代价。

5.2 RQ1: 预算分档是否形成可解释前沿

表 3 给出四个预算档位在双基准上的主结果。主结果统一使用 `claude-sonnet-4-6`，报告三随机种子（42、78、132）的均值与样本标准差。策略列使用第 3.2 节的语义档位，括号中只保留实现模式标

识，便于复现。表中边际 ρ 相对上一档计算，单位为百分点/万令牌；当相邻两档令牌不增时不定义。

表 3 显示，MBPP 上四档通过率从 82.51% 连续提升到 91.72%，说明预算扩展能稳定换取更高上限；HumanEval 上单步已达 97.56%，后续增益主要来自少量残差样本，因此边际收益波动更明显。

图 5 将主结果放到统一成本-通过率坐标中，以便观察两个基准在不同动态范围下的前沿形态。

图 5 进一步表明，单步 + 修补的边际 ρ 最高，是最具性价比的第一层增益；分流救援与级联继续提高上限，并对应更高令牌成本。两组结果共同说明，预算分档的收益与任务分布动态范围密切相关。

5.3 RQ2：分档形态是否跨模型保持一致

表 4 先报告 MBPP 上多个基础模型在四档协议下的通过率，用于观察分档形态在较宽动态范围任务上的可迁移性。该表采用随机种子 42 单次运行，数值单位为通过率百分比。

表 4: MBPP 多模型预算分档结果

模型	单步	单步 + 修补	分流 + 救援	级联
claude-sonnet-4-6	82.51	86.65	88.52	91.72
gpt-5.4	72.13	77.75	79.39	85.48
gpt-5.3-codex	71.90	78.92	79.39	83.84
qwen3-coder-plus	71.90	76.58	79.63	81.97
qwen3-30b-a3b-instruct-2507	68.38	75.18	77.99	80.80
gemini-3.1-flash-lite-preview	73.54	76.81	79.39	80.09
qwen-plus	70.73	75.18	77.28	79.63

MBPP 结果显示，所有模型从单步到单步 + 修补均有提升，多数模型在分流和级联档继续上升。这说明执行反馈修补是跨模型稳定收益来源，而候选扩展更偏向提升高预算上限。

表 5 进一步报告 HumanEval 上的多模型结果。与 MBPP 相比，HumanEval 在强模型下更接近饱和，因此更适合观察高预算策略是否仍能处理少量残差样本。

表 5: HumanEval 多模型预算分档结果

模型	单步	单步 + 修补	分流 + 救援	级联
claude-sonnet-4-6	97.56	99.19	99.39	99.80
gemini-3.1-flash-lite-preview	97.56	98.78	99.39	98.78
gpt-5.3-codex	95.12	97.56	98.78	99.39
gpt-5.4	95.12	97.56	97.56	98.78
qwen3-coder-plus	95.73	97.56	98.78	98.17
qwen3-30b-a3b-instruct-2507	92.68	95.73	97.56	96.34
qwen-plus	92.68	97.50	96.34	95.12

HumanEval 上整体接近饱和，部分模型在高档出现回落或持平，表明高预算级联更适合承担上限探索，而中预算修补和分流档更适合常规部署。综合两表，预算分档的粗形态具有跨模型一致性，但高档收益会受到基准饱和程度影响。

5.4 RQ3：关键机制如何影响前沿

本节分析三类机制：结构路由、反馈回注和模块开关。除特别说明外，细粒度消融多在 MBPP 上以随机种子 42 单次运行；MBPP 的动态范围更宽，更适合承载模块排序和成本前沿分析。

5.4.1 结构分的预算先验作用

本文先检验静态结构分与运行失败之间的关系。在任务级样本上使用逻辑回归 5 折交叉验证时，首轮失败与需重路由两类标签的 ROC-AUC 分别约为 0.471 与 0.473；加入扩展静态特征后，AUC 约升至 0.51。结果表明，结构分的价值主要体现在预算排序：它把结构更复杂的题目优先送入重路径，再由端到端前沿、阈值扫描和路由对照共同体现收益。

5.4.2 固定多重集随机路由对照

表 6 报告 difficulty_aware_plus 档在 $b=0.6$ 、 $\tau=1.6$ 下的配对结果，模型为 claude-sonnet-4-6，采用随机种子 42 单次运行。结构路由为默认设置；随机对照为第 4.2 节定义的固定多重集随机置换。两者在同一数据集上的轻/重路径数量完全一致，因此比较的是同一重路径覆盖率下，题目与槽位对齐方式的影响。

表 3: 预算分档编排主结果

数据集	策略	通过率 (%)	平均调用次数	总令牌	边际 ρ
MBPP	单步 (single_step)	82.51 ± 0.48	1.000 ± 0.000	$113,922 \pm 5,621$	—
MBPP	单步 + 修补 (single_step_repair)	86.65 ± 0.24	1.177 ± 0.001	$141,207 \pm 4,393$	1.52
MBPP	分流 + 重链与救援 (difficulty_aware_plus)	88.52 ± 0.24	1.483 ± 0.009	$218,327 \pm 2,138$	0.24
MBPP	级联 (workflow_cascade)	91.72 ± 0.36	1.639 ± 0.021	$321,177 \pm 11,854$	0.31
HumanEval	单步 (single_step)	97.56 ± 0.00	1.000 ± 0.000	$64,938 \pm 116$	—
HumanEval	单步 + 修补 (single_step_repair)	99.19 ± 0.35	1.020 ± 0.003	$66,925 \pm 1,588$	8.19
HumanEval	分流 + 重链与救援 (difficulty_aware_plus, v2, $\tau=1.6$)	99.39 ± 0.00	1.067 ± 0.010	$75,151 \pm 1,064$	0.25
HumanEval	级联 (workflow_cascade, 默认基 线)	99.80 ± 0.35	1.055 ± 0.010	$73,847 \pm 1,422$	—

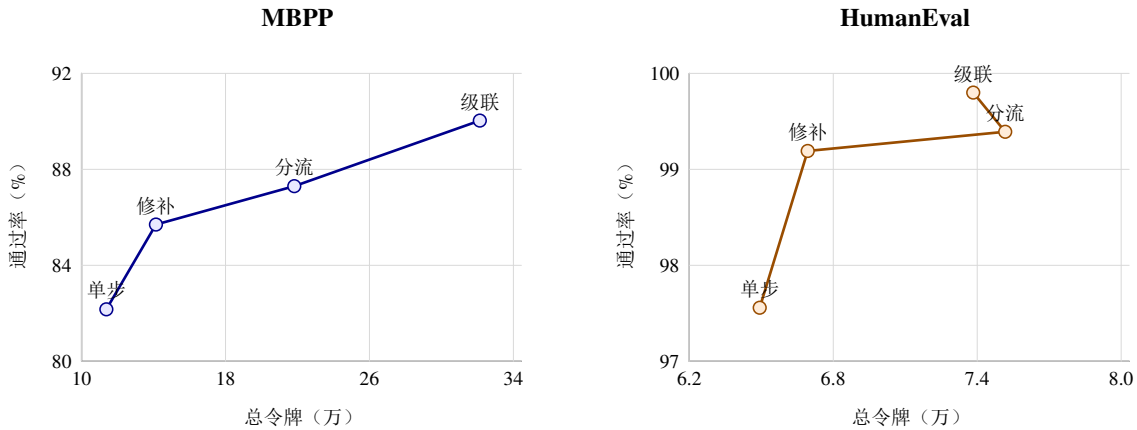


图 5: 预算分档的成本-通过率前沿

表 6: 固定多重集随机路由对照

数据集	结构 通过	随机 通过	结构 令牌	随机 令牌
MBPP	383/427 89.70%	379/427 88.76%	267,878	234,912
HumanEval	163/164 99.39%	163/164 99.39%	77,683	76,563

在本次配对中，MBPP 上结构路由多通过 4 题，平均调用次数为 677 次，高于随机对照的 640 次；HumanEval 上通过数相同，平均调用次数分别为 178 次和 177 次，令牌差异也较小。结果显示，在固定重路径覆盖率下，结构对齐能在更复杂分布上带来额外收益，并形成相应成本。

5.4.3 反馈回注形态

表 7 比较压缩反馈与完整回溯。该实验在 MBPP 427 上使用 difficulty_aware_plus 档与 $b = 0.6$ ，报告五随机种子的均值与样本标准差；除修补阶段回注形态外，其余模型、预算、任务与路由配置保持一致。

表 7: E2 反馈形态对照

回注形态	通过数	pass@1(%)	总令牌
压缩回注	379.4 ± 3.85	88.852 ± 0.900	$229,713 \pm 7,558$
完整回溯	379.0 ± 1.41	88.756 ± 0.334	$233,180 \pm 2,781$

两组通过率均值差约 0.096 个百分点，低于各

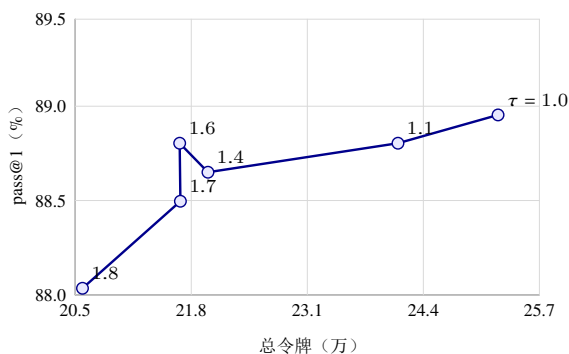


图 6: 结构阈值的精度-成本变化

自标准差量级；令牌均值差约 3,467。压缩回注以更短上下文取得相近通过率，适合作为默认回注形态。

5.4.4 阈值、主链与级联开关

表 8 在 MBPP 427 上使用 v2 打分器和随机种子 42 单次运行，扫描结构阈值 τ 。较低阈值提高重路径覆盖率和令牌成本，较高阈值降低成本并减少重路径样本。

表 8: 分流 + 重链与救援档阈值消融

阈值 τ	轻/重路径	pass@1	平均调用	总令牌
$\tau=1.0$	92/335	382/427	1.646	255,523
$\tau=1.1$	127/300	381/427	1.597	243,529
$\tau=1.4$	187/240	380/427	1.511	220,825
$\tau=1.6$	213/214	381/427	1.489	217,510
$\tau=1.7$	224/203	379/427	1.476	217,534
$\tau=1.8$	254/173	376/427	1.447	205,909

阈值扫描表明， $\tau = 1.0$ 获得最高通过数，但代价是更多重路径样本与最高令牌消耗； $\tau = 1.8$ 成本最低，却损失 6 个通过样本。默认 $\tau = 1.6$ 位于二者之间，在通过数与成本之间形成较稳妥折中。

图 6 展示同一扫描中的精度-成本位置，默认阈值位于较平衡的区间。

表 9 在 MBPP 427 上以 $\tau = 1.6$ 和随机种子 42 单次运行，关闭分流 + 重链与救援档中的关键模块。结果显示，结构化主链对通过率影响最大；候选救援也有收益，但带来主要额外成本；精修救援边际较小。

表 9: 分流 + 重链与救援档流程消融

配置	pass@1	平均调用	总令牌
完整流程	381/427	1.489	217,510
-hard_spec	360/427	1.164	148,549
-hard_cot	375/427	1.436	202,847
-plus_candidate	377/427	1.283	167,780
_rescue	378/427	1.450	207,200
-plus_refine	378/427	1.450	207,200
_rescue	378/427	1.450	207,200

流程消融显示，去掉结构化主链后通过数从 381 降到 360，是最明显的性能损失；去掉候选救援可显著节省令牌，但也损失 4 个通过样本。精修救援的边际贡献较小，更像成本-精度微调项。

表 10 在 MBPP 427 上以随机种子 42 单次运行，聚焦级联档。移除候选扩展后通过率大幅下降，说明候选扩展是高预算档的核心；精修相关开关主要调节通过率与令牌成本。

表 10: 级联档流程消融

配置	pass@1	平均调用	总令牌
主配置	392/427	1.595	299,935
-extra	355/427	1.000	108,610
-refine_once	387/427	1.529	280,307
+semantic	390/427	1.541	280,784
_refine_gate	390/427	1.541	280,784

级联消融进一步说明，高预算档的主要收益来自候选扩展：关闭 -extra 后通过数下降 37 题。限制精修次数或加入语义门控只造成小幅降分，但可明显降低令牌成本，因此更适合作为部署时的成本控制开关。

5.5 RQ4: 成本-收益前沿

表 11 报告相对单步档的累计收益率，单位为每万令牌提升百分点。它补充表 3 的边际 ρ ，用于观察从最低成本基线到各档的整体收益。

表 11: 预算分档的经验收益率前沿

数据集	档位	Δp	Δt	$\rho_{cum} (\times 10^4)$
MBPP	单步 + 修补	4.14	27,284	1.52
MBPP	分流 + 重链	6.01	104,405	0.58
MBPP	与救援	9.21	207,255	0.44
HumanEval	级联	1.63	1,986	8.19
HumanEval	单步 + 修补	1.83	10,213	1.79
HumanEval	分流 + 重链	2.24	8,909	2.51
HumanEval	与救援			
HumanEval	级联			

MBPP 上, 累计 ρ 从单步 + 修补到级联逐步下降, 说明后续预算仍能换取通过率, 但边际性价比降低。HumanEval 上单步已经接近饱和, 少量残差样本会放大个别题目对 ρ 的影响。综合来看, 中预算优先使用修补与结构化主链, 高预算再启用候选扩展, 是更稳妥的部署选择。

6 讨论

6.1 预算分档的适用边界

预算分档编排的收益取决于任务分布、基础模型能力和成本约束之间的匹配关系。MBPP 题面更复杂、单步残差更大, 修补、分流救援和级联能够逐级抬高通过率; HumanEval 已接近饱和, 后续档位主要收束少量残差题。两组结果共同说明, 预算分档的价值来自按任务动态范围配置计算投向。

级联档的定位也更清晰。它在 MBPP 上提供最高通过率, 同时消耗更多令牌; 在 HumanEval 或部分模型上, 级联相对中档的增益较小。实际部署中, 级联适合高价值样本、离线批处理或追求上限的场景; 在线常规场景则更适合以单步 + 修补和分流主链作为默认路径。

6.2 机制证据的解释

学习式路由基线显示, 当前静态结构特征对运行失败标签的直接判别能力接近随机; 这并不削弱结构分的作用, 反而说明它更适合作为预算排序信号。结构分把题面复杂、接口复杂或测试条件密集的样本优先送入重路径, 端到端前沿、阈值消融和固定多重集对照共同支撑了这一用法。

固定多重集随机路由对照进一步拆开了覆盖率与结构对齐。该实验固定轻/重路径数量, 只改变题目与槽位的对应关系。MBPP 上结构路由多通过 4 题但消耗更多令牌, 说明结构对齐带来可见收益, 也对应更高计算投入; HumanEval 上通过数相同, 则与其饱和分布一致。

反馈形态对照给出了明确的工程结论。五随机种子结果中, 压缩回注与完整回溯取得相近通过率, 且平均令牌更低。压缩回注保留错误类型、关键片段和定向修补提示, 减少了无关运行栈文本, 是本文中预算档位的默认反馈形式。

6.3 部署建议

基于主结果与消融, 预算策略可以分为两层。中预算场景下, 优先保留单步 + 修补、结构化主链与压缩失败回注, 这些模块在 MBPP 上具有较好的成本-收益关系, 也更容易解释: 它们把失败信息转化为结构化约束, 帮助模型完成局部修补或更聚焦的重链生成。

高预算场景下, 新增计算应优先投入候选扩展, 再视成本约束开启精修。级联消融显示, 候选扩展是高预算档的核心增益来源; 精修相关模块更多承担细粒度折中。若预算只允许增加一个高档模块, 候选扩展通常比后处理精修更值得保留。

落地前仍需在目标业务数据上做小规模回归。简单模板题会削弱级联收益, 复杂边界条件、长测试或多约束签名会放大分流与候选扩展的价值。预算分档协议的优势在于可以用同一记账口径快速完成这种回归。

6.4 局限与未来工作

本文仍有若干局限。细粒度消融以 MBPP 为主; HumanEval 更适合作为残差收束观察。多模型表与多数消融表采用随机种子 42, 用于展示形态和机制趋势; 固定多重集路由对照目前也是单次配对, 后续可扩展为多随机种子实验。

此外, 本文将总令牌作为接口成本主代理, 真实部署还会考虑按次计费、延迟、并发限制与失败重试策略。未来工作将把 ρ 扩展为包含固定调用成本、延迟惩罚和预算上限的联合指标, 并持续跟踪模型版本更新带来的单步能力、反馈敏感性和候选多样性变化。

7 结论

本文围绕函数级代码生成中的测试时计算组织问题, 提出预算分档编排协议。该协议在统一执行器、统一判定口径和统一令牌记账下, 将流程固定为单步、单步 + 修补、分流 + 重链与救援、级联四个可比较档位, 把算得更多转化为预算投向何处的可复现实验问题。

HumanEval 与 MBPP 双基准上的三随机种子主结果显示, 分档编排形成了清晰的成本-收益前沿: MBPP 通过率从 82.51% 提升至 91.72%, HumanEval

从 97.56% 提升至 99.80%。机制实验表明，结构难度分适合作为预算先验，压缩反馈是经济有效的回注形式，级联档的主要收益来自候选扩展。

总体而言，预算分档编排提供了一种面向部署的代码生成研究口径。中预算优先修补与结构化主链，高预算优先候选扩展，是当前数据下最有解释力的策略。后续工作将扩展多随机种子路由对照、跨模型版本回归以及更贴近真实部署成本的联合记账模型。

A 难度子项与权重依据的补充统计

表 12 汇总本文已使用结果文件中的代表性统计，数据来自 MBPP 上的离线路由证据与特征实验报告：一类是子项与结构分档/运行标签的 Spearman 相关，另一类是学习式路由基线在不同特征块上的交叉验证 AUC。该表用于支撑第 5.4 节关于结构分主要承担预算排序先验、而非运行失败分类器的判断。

表 12: 难度子项与权重依据的补充统计

统计项	指标	数值	<i>n</i>
$\rho(\text{structural_hard, test_score})$	Spearman	0.5696	427
$\rho(\text{direct_fail, test_score})$	Spearman	-0.0337	383
$\rho(\text{direct_fail, boundary_score})$	Spearman	0.0483	383
$\rho(\text{direct_fail, extent_score})$	Spearman	0.0669	383
need_reroute (v2 五子分)	LR 5-fold AUC	0.450 ± 0.103	383
need_reroute (扩展静态)	LR 5-fold AUC	0.520 ± 0.108	383

该补充表支持本文的方法定位：结构难度与测试复杂度的相关系数为 0.5696，说明测试复杂度能够进入结构先验；而首轮失败与各静态子分的相关性接近 0，LR 5 折 AUC 也仅从 0.450 提升到 0.520。由此可见，难度子项更适合作为可解释的预算分配先验，而非直接充当运行失败分类器。主结论仍以端到端通过率、调用与令牌前沿为准。

参考文献

- [1] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code[EB/OL]. arXiv:2107.03374, 2021.
- [2] AUSTIN J, ODENA A, NYE M, et al. Program synthesis with large language models[EB/OL]. arXiv:2108.07732, 2021. DOI:10.48550/arXiv.2108.07732.
- [3] LI Y, CHOI D, CHUNG J, et al. Competition-level code generation with AlphaCode[J]. Science, 2022, 378(6624): 1092-1097. DOI:10.1126/science.abq1158.
- [4] CHEN X, LIN M, SCHARLI N, et al. Teaching large language models to self-debug[C/OL]//The Twelfth International Conference on Learning Representations. 2024.
- [5] SHINN N, CASSANO F, BERMAN E, et al. Reflexion: language agents with verbal reinforcement learning[C/OL]//Advances in Neural Information Processing Systems 36. 2023.
- [6] GOU Z, SHAO Z, GONG Y, et al. CRITIC: large language models can self-correct with tool-interactive critiquing[C/OL]//The Twelfth International Conference on Learning Representations. 2024.
- [7] LI D, CAO S, CAO C, et al. S*: test time scaling for code generation[C]//Findings of the Association for Computational Linguistics: EMNLP 2025. Suzhou: Association for Computational Linguistics, 2025: 15964-15978. DOI:10.18653/v1/2025.findings-emnlp.865.
- [8] HENDRYCKS D, BASART S, KADAVATH S, et al. Measuring coding challenge competence with APPS[C]//Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. 2021.
- [9] LIU J, XIA C S, WANG Y, et al. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code genera-

- tion[C]//Advances in Neural Information Processing Systems 36. 2023.
- [10] JAIN N, HAN K, GU A, et al. LiveCodeBench: holistic and contamination free evaluation of large language models for code[EB/OL]. arXiv:2403.07974, 2024.
- [11] LAI Y, LI C, WANG Y, et al. DS-1000: a natural and reliable benchmark for data science code generation[C]//Proceedings of the 40th International Conference on Machine Learning. 2023: 18319-18345.
- [12] HUANG D, ZHANG J M, LUCK M, et al. AgentCoder: multi-agent-based code generation with iterative testing and optimisation[EB/OL]. arXiv:2312.13010, 2024. DOI:10.48550/arXiv.2312.13010.
- [13] MADAAN A, TANDON N, GUPTA P, et al. Self-Refine: iterative refinement with self-feedback[C/OL]//Advances in Neural Information Processing Systems 36. 2023.
- [14] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems 35. 2022.
- [15] WANG X, WEI J, SCHUURMANS D, et al. Self-consistency improves chain of thought reasoning in language models[C/OL]//The Eleventh International Conference on Learning Representations. 2023.
- [16] YAO S, ZHAO J, YU D, et al. ReAct: synergizing reasoning and acting in language models[C/OL]//The Eleventh International Conference on Learning Representations. 2023.
- [17] YAO S, YU D, ZHAO J, et al. Tree of thoughts: deliberate problem solving with large language models[C]//Advances in Neural Information Processing Systems 36. 2023.
- [18] SCHICK T, DWIVEDI-YU J, DESSI R, et al. Toolformer: language models can teach themselves to use tools[C]//Advances in Neural Information Processing Systems 36. 2023.
- [19] LU S, GUO D, REN S, et al. CodeXGLUE: a machine learning benchmark dataset for code understanding and generation[C]//Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. 2021.
- [20] CHEN B, ZHANG F, NGUYEN A, et al. CodeT: code generation with generated tests[C/OL]//The Eleventh International Conference on Learning Representations. 2023.
- [21] LE H, WANG Y, GOTMARE A D, et al. CodeRL: mastering code generation through pre-trained models and deep reinforcement learning[C]//Advances in Neural Information Processing Systems 35. 2022.
- [22] ADNAN M, XU Z, KUHN C C N. Large language model guided self-debugging code generation[EB/OL]. arXiv:2502.02928, 2025.
- [23] CHEN X, TAO Z, ZHANG K, et al. Revisit self-debugging with self-generated tests for code generation[C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025: 18003-18023. DOI:10.18653/v1/2025.acl-long.881.